

Contesto del Progetto

Il progetto PlatformAI è stato costruito come piattaforma di base per lo studio applicato dell'intelligenza artificiale in contesti industriali. L'architettura attuale copre già i concetti fondamentali: streaming SSE con Azure OpenAI, tool calling, training ML incrementale con checkpoint, sistema multi-tenant e gestione utenti. Questo documento descrive i prossimi passi consigliati, ordinati per priorità didattica e impatto pratico sull'applicativo.

Funzionalità già implementate

AI Assistant	Chat con LLM via Azure OpenAI, streaming SSE token-by-token, stop streaming.
Tool Calling	Il modello chiama autonomamente tool (get_production_data, get_prediction) per rispondere con dati reali.
Training ML Incrementale	Checkpoint-based training con ML.NET, configurazione parametrizzata, seed dati produzione.
Analytics Streaming	Grafici ApexCharts generati dinamicamente dalla risposta AI, card predizione ML.
Multi-tenant	Isolamento dati per tenant, contesto tenant iniettato nel system prompt.
Gestione Utenti	CRUD completo con ruoli, abilita/disabilita, reset password.

Prossimi Passi Consigliati

1 RAG – Retrieval-Augmented Generation

Il passo più importante per estendere le capacità del modello. Attualmente l'AI risponde solo con i dati che le vengono passati via tool. Con RAG è possibile caricare documenti (manuali macchine, SOP, report tecnici) e farglieli "ricordare" tramite ricerca vettoriale semantica.

Concetti da studiare:

- Embedding: come i testi vengono convertiti in vettori numerici
- Vector store: pgvector (PostgreSQL) oppure Qdrant come database vettoriale
- Chunking strategy: come suddividere i documenti in frammenti ottimali
- Similarity search: cosine distance, MMR (Maximum Marginal Relevance)

Integrazione nel progetto:

Aggiungere un nuovo tool `search_documents(query)` che il modello chiama quando ha bisogno di consultare la documentazione. Il tool esegue una ricerca semantica nel vector store e restituisce i chunk più rilevanti come contesto.

2 Agent Loop Multi-Step (ReAct)

Oggi il servizio `LLMAalyticsService` esegue un singolo ciclo di tool calling. Il passo successivo è implementare un loop completo: il modello chiama un tool, osserva il risultato, decide se chiamarne un altro, e continua fino alla risposta finale.

Esempio di query che lo richiede:

"Confronta il rendimento di questa settimana con la stessa settimana del mese scorso e spiegami perché è diverso" — richiede: `get_data(week_current)` → `get_data(week_last_month)` → analisi comparativa.

Pattern da studiare:

- ReAct (Reasoning + Acting): il modello ragiona su cosa fare prima di agire
- Tool call loop con `FinishReason.ToolCalls` già presente nel codice come punto di partenza
- Gestione del contesto: come mantenere la catena di pensiero tra più chiamate

3 Evaluation Framework (Evals)

Senza metriche oggettive non è possibile sapere se una modifica al prompt, al modello o alla pipeline ha migliorato o peggiorato la qualità delle risposte. Gli evals sono il fondamento di qualsiasi sviluppo AI serio.

Metriche principali da misurare:

- Faithfulness: la risposta è coerente con i dati forniti come contesto?
- Answer relevancy: la risposta è pertinente alla domanda?
- Hallucination rate: il modello inventa dati che non ha ricevuto?
- Tool call accuracy: il modello chiama il tool giusto con i parametri corretti?

Tool consigliati:

- Promptfoo: framework open-source per testare prompt con dataset
- RAGAS: libreria Python specializzata per valutare pipeline RAG

Nel progetto PlatformAI i dati di produzione reali costituiscono già un ottimo ground truth per costruire un dataset di domande/risposte attese.

4 Observability LLM (Tracing)

Ogni chiamata al modello è una black box. Integrare un sistema di tracing permette di avere visibilità completa su ogni interazione: quali token vengono generati, quali tool vengono chiamati, dove il modello sbaglia e quanto costa ogni conversazione.

Cosa monitorare:

- Latenza per token e latenza totale per risposta
- Costo stimato (token input/output × prezzo del modello)
- Traccia completa: system prompt → tool calls → risposta finale
- Tasso di errore e tipologia di errori del modello

Tool consigliati:

- LangSmith (LangChain): tracing visuale, replay di conversazioni
- Azure AI Foundry tracing: integrazione nativa con Azure OpenAI
- OpenTelemetry: standard aperto per trace distribuiti

5 Prompt Versioning e A/B Testing

Il system prompt in `BuildMessagesWithAnalyticsContext` è attualmente hardcoded nel codice. Estrarlo su database permette di versionarlo, modificarlo senza deploy e confrontare diverse versioni sulle stesse domande.

Architettura suggerita:

- Tabella `PromptTemplate`: id, version, name, content, isActive, createdAt
- API per attivare/disattivare versioni senza riavviare il backend
- A/B testing: instradare il 50% delle richieste alla versione A e il 50% alla B
- Dashboard per confrontare metriche tra versioni (usa gli Evals del passo 3)

Questo passo insegna un concetto chiave: il prompt è codice, va versionato e testato con la stessa disciplina del codice sorgente.

6 Fine-Tuning del Modello

Con i dati di produzione accumulati è possibile creare un dataset di conversazioni reali (domanda + risposta ideale) e fare fine-tuning su Azure OpenAI. È il passo più avanzato ma anche il più potente per adattare il modello al dominio industriale specifico.

Prerequisiti prima di iniziare:

- Almeno 50-100 esempi domanda/risposta di alta qualità (meglio 500+)
- Sistema di evals funzionante per misurare il miglioramento post-tuning
- Budget Azure per il training (ordine di grandezza: decine di dollari per job base)

Formato dati per Azure OpenAI fine-tuning:

```
{"messages": [{"role": "system", "content": "..."}, {"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]}
```

Il fine-tuning permette di ridurre la lunghezza del system prompt (il modello ha già interiorizzato le istruzioni), abbassare la latenza e migliorare la qualità delle risposte dominio-specifiche.

Ordine di Implementazione Consigliato

1	RAG	Impatto immediato, risultati visibili
2	Agent Loop	Sblocca query complesse multi-step

3	Evals	Fondamentale, costruirlo presto
4	Observability	Debugging e ottimizzazione continua
5	Prompt Versioning	Disciplina ingegneristica del prompt
6	Fine-Tuning	Fase matura, richiede dataset e budget

I passi 1 e 2 danno risultati visibili subito nell'applicativo. Il passo 3 (Evals) dovrebbe essere avviato in parallelo appena possibile: è molto più difficile costruire un sistema di valutazione retroattivamente quando ci sono già molte versioni del prompt in circolazione. Fine-tuning è il traguardo finale del percorso.